



(<http://www.pieriandata.com>)

Data Input and Output

This notebook is the reference code for getting input and output, pandas can read a variety of file types using its `pd.read_` methods. Let's take a look at the most common data types:

```
In [1]: import numpy as np
import pandas as pd
```

CSV

CSV Input

```
In [25]: df = pd.read_csv('example')
df
```

```
Out[25]:
```

	a	b	c	d
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

CSV Output

```
In [24]: df.to_csv('example', index=False)
```

Excel

Pandas can read and write excel files, keep in mind, this only imports data. Not formulas or images, having images or macros may cause this read_excel method to crash.

Excel Input

```
In [35]: pd.read_excel('Excel_Sample.xlsx', sheetname='Sheet1')
```

```
Out[35]:
```

	a	b	c	d
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15

Excel Output

```
In [33]: df.to_excel('Excel_Sample.xlsx', sheet_name='Sheet1')
```

HTML

You may need to install html5lib, lxml, and BeautifulSoup4. In your terminal/command prompt run:

```
conda install lxml
conda install html5lib
conda install BeautifulSoup4
```

Then restart Jupyter Notebook. (or use pip install if you aren't using the Anaconda Distribution)

Pandas can read table tabs off of html. For example:

HTML Input

Pandas read_html function will read tables off of a webpage and return a list of DataFrame objects:

```
In [5]: df = pd.read_html('http://www.fdic.gov/bank/individual/failed/banklist.html')
```

In [7]: `df[0]`

Out[7]:

	Bank Name	City	ST	CERT	Acquiring Institution	Closing Date	Updated Date	Loss Share Type	Agr Ter
0	First CornerStone Bank	King of Prussia	PA	35312	First-Citizens Bank & Trust Company	May 6, 2016	July 12, 2016	none	
1	Trust Company Bank	Memphis	TN	9956	The Bank of Fayette County	April 29, 2016	August 4, 2016	none	
2	North Milwaukee State Bank	Milwaukee	WI	20364	First-Citizens Bank & Trust Company	March 11, 2016	June 16, 2016	none	
3	Hometown National Bank	Longview	WA	35156	Twin City Bank	October 2, 2015	April 13, 2016	none	
4	The Bank of Georgia	Peachtree City	GA	35259	Fidelity Bank	October 2, 2015	April 13, 2016	none	
					United Fidelity	July 10	July 12		

SQL (Optional)

- Note: If you are completely unfamiliar with SQL you can check out my other course: "Complete SQL Bootcamp" to learn SQL.

The `pandas.io.sql` module provides a collection of query wrappers to both facilitate data retrieval and to reduce dependency on DB-specific API. Database abstraction is provided by SQLAlchemy if installed. In addition you will need a driver library for your database. Examples of such drivers are `psycopg2` for PostgreSQL or `pymysql` for MySQL. For SQLite this is included in Python's standard library by default. You can find an overview of supported drivers for each SQL dialect in the SQLAlchemy docs.

If SQLAlchemy is not installed, a fallback is only provided for `sqlite` (and for `mysql` for backwards compatibility, but this is deprecated and will be removed in a future version). This mode requires a Python database adapter which respect the Python DB-API.

See also some cookbook examples for some advanced strategies.

The key functions are:

- `read_sql_table(table_name, con[, schema, ...])`
 - Read SQL database table into a DataFrame.
- `read_sql_query(sql, con[, index_col, ...])`

- Read SQL query into a DataFrame.
- `read_sql(sql, con[, index_col, ...])`
 - Read SQL query or database table into a DataFrame.
- `DataFrame.to_sql(name, con[, flavor, ...])`
 - Write records stored in a DataFrame to a SQL database.

```
In [36]: from sqlalchemy import create_engine
```

```
In [37]: engine = create_engine('sqlite:///memory:')
```

```
In [40]: df.to_sql('data', engine)
```

```
In [42]: sql_df = pd.read_sql('data', con=engine)
```

```
In [43]: sql_df
```

```
Out[43]:
```

	index	a	b	c	d
0	0	0	1	2	3
1	1	4	5	6	7
2	2	8	9	10	11
3	3	12	13	14	15

Great Job!